

Introducing Python

- Python is a high-level (human-readable) programming language that is processed by the Python “interpreter” to produce results.
- Python is derived from many other languages, including C, C++, the Unix shell and other programming languages
- The basic philosophy of the Python language is readability, which makes it particularly well-suited for beginners in computer programming

Python's key distinguishing features

- **Python is free – is open source distributable software**
- **Python is easy to learn – has a simple language syntax**
- **Python is easy to read – is uncluttered by punctuation**
- **Python is easy to maintain – is modular for simplicity**
- **Python is “batteries included” – provides a large standard library for easy integration into your own programs**
- **Python is interactive – has a terminal for debugging and testing snippets of code**
- **Python is portable – runs on a wide variety of hardware platforms and has the same interface on all platforms**
- **Python is interpreted – there is no compilation required**
- **Python is high-level – has automatic memory management**
- **Python is extensible – allows the addition of low-level modules to the interpreter for customization**
- **Python is versatile – supports both procedure-orientated programming and object orientated programming (OOP)**
- **Python is flexible – can create console programs, windowed GUI (Graphical User Interface) applications, and CGI (Common Gateway Interface) scripts to process web data**

Meeting the interpreter

- The Python interpreter processes text-based program code and also has an interactive
- mode where you can test snippets of code and is useful for debugging code.

Python's interactive mode can be entered in a number of ways:

1. From a regular Command Prompt – simply enter the command **'python'** to produce the Python primary prompt `>>>` where you can interact with the interpreter
2. From the Start Menu – choose “Python (command line)” to open a window containing the Python `>>>` primary prompt
3. From the Start Menu – choose “IDLE (Python GUI)” to launch a Python Shell window containing the Python `>>>` primary prompt

Using Python as a Calculator

- `>>> 2 + 2` `4`
- `>>> 50 - 5*6` `20`
- `>>> (50 - 5*6) / 4` `5.0`
- `>>> 8 / 5` *# division always returns a floating point number* `1.6`

Using Python as a Calculator

- `>>> 17 / 3 # classic division returns a float`
`5.666666666666667 >>>`
- `>>> 17 // 3 # floor division discards the`
`fractional part 5`
- `>>> 17 % 3 # the % operator returns the`
`remainder of the division 2`
- `>>> 5 * 3 + 2 # result * divisor + remainder 17`

Using Python as a Calculator

- `>>> 5 ** 2 # 5 squared` 25
- `>>> 2 ** 7 # 2 to the power of 7` 128
- `>>> width = 20`
- `>>> height = 5 * 9`
- `>>> width * height` 900

Writing your first program

- A Python program is simply a plain text file script created with an editor, such as Windows' Notepad, that has been saved with a “.py” file extension.
- Python programs can be executed by stating the script file name after the **python** command at a terminal prompt.
- In Python, the **print()** function is used to specify the message within its parentheses. This must be a string of characters enclosed between quote marks. These may be “ ” double quote marks **or** ‘ ’ single quote marks – but **not** a mixture of both.

The 'Hello World' Program

1. On Windows, launch any plain text editor such as the Notepad application
2. Next, precisely type the following statement into the empty text editor window
print('Hello World!')
3. Now, create a new directory at **D:\MyScripts** and save the file in it as **hello.py**
4. Finally, launch a Command Prompt window, navigate to the new directory and precisely enter the command **python hello.py** – to see the Python interpreter run your program and print out the specified greeting message

Employing variables

- In programming, a “variable” is a container in which a data value can be stored within the computer’s memory.
- The stored value can then be referenced using the variable’s name.
- The programmer can choose any name for a variable, except the Python keywords

NOTE:

- it is good practice to choose meaningful names that reflect the variable’s content.
- String data must be enclosed within quote marks to denote the start and end of the string.

Different ways of assigning values to variables:

1. `a = 8`
2. `a = b = c = 8`
3. `a , b , c = 1 , 2 , 3`

NOTE:

- Some programming languages, such as Java, demand you specify what type of data a variable may contain in its declaration. This reserves a specific amount of memory space and is known as “static typing”.
- Python variables, on the other hand, have no such limitation and adjust the memory allocation to suit the various data values assigned to their variables (“dynamic typing”).
- This means they can store integer whole numbers, floating-point numbers, text strings, or Boolean values of **True** or **False** as required.

Demonstrating variable declaration

Initialize a variable with an integer value.

```
var = 8
```

```
print( var )
```

Assign a float value to the variable.

```
var = 3.142
```

```
print( var )
```

Assign a string value to the variable.

```
var = 'Python in easy steps'
```

```
print( var )
```

Assign a boolean value to the variable.

```
var = True
```

```
print( var )
```

NOTE: Multi-line comments can be added to a script if enclosed between triple quote marks `""" ... """` .

Accepting user input 1

Initialize a variable with a user-specified value.

```
user = input( 'I am Python. What is your name? : ' )
```

Output a string and a variable value.

```
print( 'Welcome' , user )
```

Accepting user input 2

```
# Initialize a variable with a user-specified value.  
user = input( 'I am Python. What is your name? : ' )
```

```
# Output a string and a variable value.  
print( 'Welcome' , user )
```

```
# Initialize another variable with a user-specified value.  
lang = input( 'Favorite programming language? : ' )
```

```
# Output a string and a variable value.  
print( lang , 'Is' , 'Fun' , sep = ' * ' , end = '\n' )
```

Correcting errors

In Python programming there are three types of error that can occur

1. **Syntax Error** – occurs when the interpreter encounters code that does not conform to the Python language rules. For example, a missing quote mark around a string.
2. **Runtime Error** – occurs during execution of the program, at the time when the program runs. For example, when a variable name is later mis-typed so the variable cannot be recognized.
3. **Semantic Error** – occurs when the program performs unexpectedly. For example, when order precedence has not been specified in an expression.

Programming errors are often called “bugs” and the process of tracking them down is often called “debugging”.

Challenges

- Convert Minutes to Seconds.
- Write a Python program to get the volume of a sphere with radius 6.
- Write a Python program which accepts the user's first and last name and print them in reverse order with a space between them.
- Program to print ASCII Value of a character
- Write a Python program that accepts an integer (n) and computes the value of $n+nn+nnn$ (e.g. $2+22+222$)